ДЗ3. Технология MapReduce

Вдовина Анастасия Романовна, ИИИ-3-ИнжДан-1-3.
Вариант 7: «Программист, разработчик на Go». 21.10.2026.

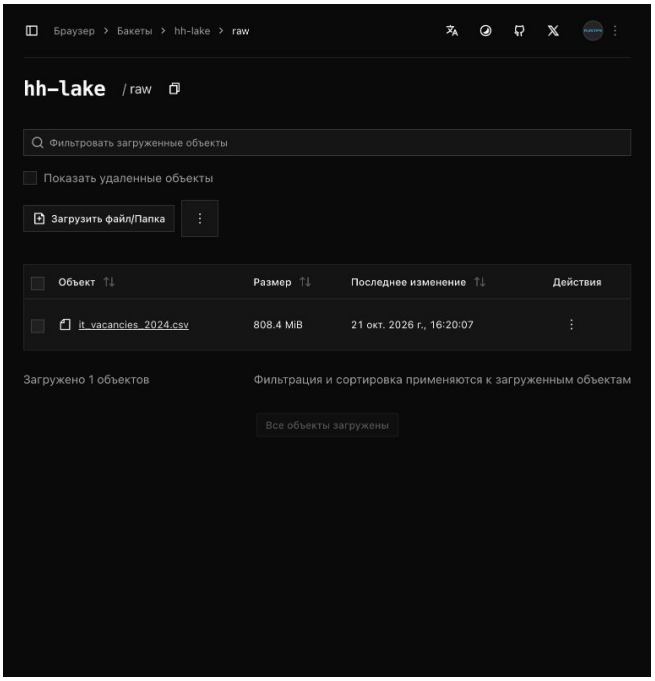
1. Запуск RustFS и загрузка данных

Запустила RustFS 1.0.1 с четырьмя папками-дисками и схемой EC 2+2, как в ДЗ2. S3 доступен на порту 9000, консоль — на 9001. Создала бакет hh-lake и загрузила полный it_vacancies_2024.csv: 229 388 записей, 23 колонки, 847 619 384 байта.

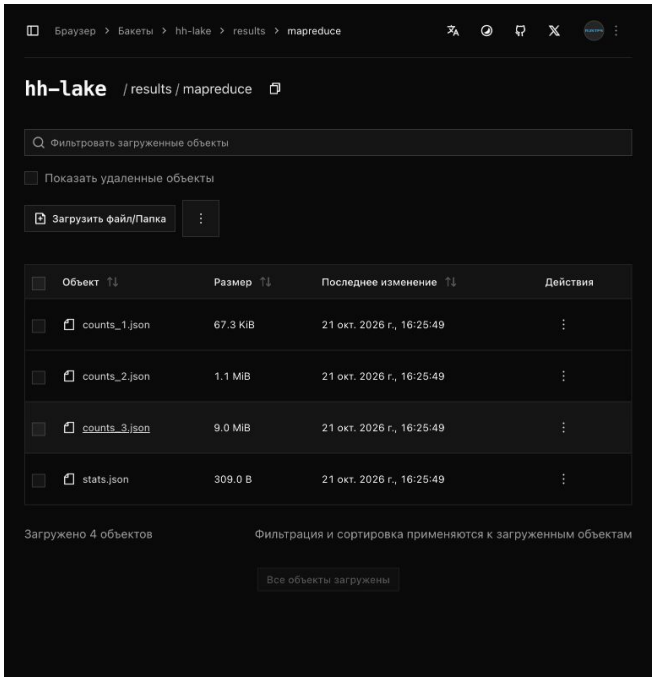
```
python -m src.infrastructure start --binary /path/to/rustfs
python -m src.infrastructure upload --source /path/to/data.csv
PUT s3://hh-lake/raw/it_vacancies_2024.csv 200
GET: bytes=847619384; SHA-256 matches source
```

После загрузки прочитала объект обратно через GET. Размер и SHA-256 совпали с исходником из ДЗ2. Контрольная сумма:

```
303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65
```



Исходный CSV в hh-lake/raw/



Результаты для k=1, 2, 3

2. Отбор своего варианта

Отбирала роль «Программист, разработчик» и Go/Golang как отдельное слово в названии или точный навык. Получила 3 123 вакансии, все vacancy_id различны. Навыки разделены через «;»: убрала пробелы и повторы, привела к нижнему регистру, объединила Go и Golang.

В 659 вакансиях навыки не перечислены. Они остаются в общем числе вакансий и статистике регионов, но сочетаний не дают. CSV читала штатным парсером: внутри описаний есть переносы строк в кавычках.

Результаты обоих способов и таблицы анализа сохранила в hh-lake/results/. Обратный GET каждого файла совпал с локальным. Проверки и SHA-256 лежат в evidence/source.json и evidence/s3_results.json. Папки-диски находятся на одном SSD — это локальный учебный стенд.

3. MapReduce и проверка через DuckDB

MapReduce читает CSV напрямую из S3 с буфером 1 МиБ, собирает чанки по 2 000 записей и отправляет их четырём процессам. В очереди не более восьми задач. Map формирует сочетания навыков, комбайнер суммирует их внутри чанка, координатор группирует ключи и reduce складывает частоты. Полный файл в память не загружается.

```
for row in rows:
    if not is_variant(row):
        continue
    skills = normalize_skills(row['key_skills'])
    for k in (1, 2, 3):
        outputs[k - 1].update(combinations(skills, k))

for destination, values in zip(counts, result[0]):
    destination.update(values)
```

В DuckDB отдельно написала SQL-фильтр варианта. Он читает тот же S3 CSV через httpfs, разбивает навыки в массив и строит пары/тройки соединением по vacancy_id с условиями a.skill < b.skill < c.skill. Так перестановки не считаются повторно. Полные запросы — в sql/.

```
WHERE role = 'Программист, разработчик'
  AND (list_contains(skills, 'go') OR
       regexp_matches(lower(name),
       '(^|[^a-za-яё0-9])(go|golang)([^a-za-яё0-9]|$)'))
```

Сравнила все сочетания и каждую частоту, а не только итоговые суммы. Оба способа выбрали 3 123 вакансии и дали одинаковые результаты:

k	Ключей MapReduce	Ключей DuckDB	Вхождений MapReduce	Вхождений DuckDB	Расхождений
1	969	969	15 519	15 519	0
2	13 552	13 552	61 694	61 694	0
3	89 260	89 260	216 333	216 333	0

Для m навыков получается C(m,k) сочетаний. Например, у вакансии с 30 навыками — 30 одиночных навыков, 435 пар и 4 060 троек. Поэтому суммарное число вхождений пар выросло в 3,98 раза относительно одиночных навыков, троек — ещё в 3,51 раза.

Проверка ошибки маппера

В отдельном запуске вызвала исключение во втором маппере до возврата результата. Координатор повторил его чанк, после чего частоты снова полностью совпали. Результат добавляется только после успешного завершения, поэтому повтор не удваивает счётчики.

```
python -m src.mapreduce --inject-failure --output work/failure
chunk=1 attempt=1 failed; retry=2
rows=229388; selected_rows=3123; chunks=115; retries=1
python -m src.verify results/mapreduce work/failure
k=1,2,3: differences=0
```

Это проверка ошибки задачи, а не отказа целой машины. Полные выводы: evidence/mapper_failure.log и failure_verification.json. Автотесты проверяют фильтр, сочетания, комбайнер, многострочный CSV, ранний отбор и независимый SQL.

4. Время, память и оптимизация

Каждый режим запустила три раза в новом процессе. Во всех запусках читался один S3 CSV, записывались JSON и проверялись все частоты. Среда: macOS arm64, Python 3.14.4, 24 ГиБ RAM, 12 логических CPU. В MapReduce четыре процесса, в DuckDB 1.4.3 — четыре потока.

Режим	Попытка	Время, с	Пик RSS, МиБ
MapReduce	1	3.164	287.9
С комбайнером	1	3.157	288.1
DuckDB	1	1.471	294.0
Ранний отбор	1	3.117	157.4
MapReduce	2	3.130	272.5
С комбайнером	2	3.140	273.5
DuckDB	2	1.192	284.6
Ранний отбор	2	3.188	160.7
MapReduce	3	3.181	266.2
С комбайнером	3	3.175	271.5
DuckDB	3	1.222	280.5
Ранний отбор	3	3.218	158.2

Медианы трёх попыток

Режим	Время, с	Пик RSS, МиБ
MapReduce	3.164	272.5
С комбайнером	3.157	273.5
DuckDB	1.222	284.6
Ранний отбор	3.188	158.2

Измеряла полное время процесса: импорт библиотек, чтение, расчёт и запись JSON. RSS опрашивался каждые 10 мс и суммировался по координатору и мапперам; для DuckDB — по одному процессу. Сервер RustFS и служебный resource tracker не учитывались. Общие страницы могут входить в RSS нескольких процессов; короткие пики между опросами могли быть пропущены.

Кэш ОС не сбрасывала, режимы чередовала. Расширение httpfs установила заранее. Все исходные замеры — в evidence/profile_runs.json, выводы запусков — в evidence/*.log.

```
python -m src.benchmark --trials 3
```

DuckDB быстрее базового MapReduce в 2.59 раза. На этом объёме SQL дешевле Python-парсинга и передачи объектов между процессами. Комбайнер уменьшил число передаваемых ключей с 293 546 до 244 531, но заметного ускорения не дал.

Для дополнительной оптимизации перенесла отбор Go-вакансий перед отправкой в процессы и поставила чанк 200 отобранных записей. Теперь передаются 3 123 записи вместо 229 388. Полный CSV всё равно читается из S3. Медианный пик RSS снизился с 272.5 до 158.2 МиБ, на 41.9%. Время осталось примерно тем же; подтверждённый выигрыш — в памяти.

5. Навыки и зарплата

Для зарплат взяла только RUR и gross=false, чтобы сравнивать рублёвые предложения на руки. Если есть обе границы, брала середину вилки, если одна — её. Получилось 622 вакансии (19,9% выборки), общая медиана — 239 750 руб..

У пары базой выбрала более частый навык, у тройки — наиболее частую вложенную пару. При равенстве частот выбирала по алфавиту. В скобках у стека и базы указала их частоты среди всех вакансий. Медиана базы включает все вакансии с этой базой, в том числе целевой стек. Порог — минимум 10 зарплат у стека и базы. Разница = (медиана стека / медиана базы - 1) × 100%.

Сочетания с более высокой медианой

k	Сочетание	База	n зарплат стек/база	Медиана тыс. руб.	База тыс. руб.	Изм., %
2	API + MySQL (32)	MySQL (302)	11/117	350.0	200.0	+75.0%
2	Backend + PHP (19)	PHP (297)	11/98	350.0	200.0	+75.0%
2	GitHub + ООП (24)	ООП (152)	24/83	362.5	215.0	+68.6%
3	API + Backend + Docker (24)	Backend + Docker (85)	10/30	350.0	200.0	+75.0%
3	API + Go + MySQL (32)	Go + MySQL (289)	11/107	350.0	200.0	+75.0%
3	Backend + Docker + PHP (16)	Docker + PHP (104)	10/33	350.0	200.0	+75.0%

Сочетания с более низкой медианой

k	Сочетание	База	n зарплат стек/база	Медиана тыс. руб.	База тыс. руб.	Изм., %
2	gRPC + MongoDB (34)	gRPC (246)	10/77	110.0	239.8	-54.1%
2	REST + SQL (53)	SQL (563)	15/149	115.0	239.8	-52.0%
2	JavaScript + PostgreSQL (43)	PostgreSQL (1117)	22/295	137.5	250.0	-45.0%
3	Git + JavaScript + PostgreSQL (27)	Git + PostgreSQL (311)	12/130	95.0	225.0	-57.8%
3	Go + gRPC + MongoDB (34)	Go + gRPC (243)	10/76	110.0	239.8	-54.1%
3	Go + REST + SQL (49)	Go + SQL (545)	11/138	120.0	240.0	-50.0%

API + MySQL даёт 350 тыс. руб. против 200 тыс. руб. у MySQL: +75%, но всего по 11 зарплатам. У тройки API + Backend + Docker те же +75% относительно Backend + Docker. Среди отрицательных различий: gRPC + MongoDB — -54,1%, Git + JavaScript + PostgreSQL — -57,8% к своей базовой паре.

Это связь в данных, а не доказательство, что конкретный навык повышает или понижает зарплату. На результат могут влиять грейд, город и работодатель. Зарплату раскрыли немногие, а группы иногда состоят всего из 10–12 наблюдений. Все сочетания с частотами и обеими медианами сохранены в results/analysis/salary_effects.csv.

6. Сравнение регионов

Выбрала три города с наибольшим числом вакансий варианта. Считала все вакансии, включая строки без навыков. Регион — поле area, зарплата — по тем же правилам, что выше.

Регион	Вакансий	С навыками	n зарплат	Медиана, руб.
Москва	1861	1436	323	270 000
Санкт-Петербург	453	371	86	250 000
Екатеринбург	102	76	24	195 000

Самые частые навыки, пары и тройки

Регион	k	Сочетание	Частота	Доля	n зарплат	Медиана тыс. руб.
Москва	1	Go	1368	73.5%	261	250.0
Москва	2	Go + PostgreSQL	623	33.5%	134	275.0
Москва	3	Docker + Go + PostgreSQL	255	13.7%	59	300.0
Санкт-Петербург	1	Go	359	79.2%	73	250.0
Санкт-Петербург	2	Go + PostgreSQL	164	36.2%	41	265.0
Санкт-Петербург	3	Docker + Go + PostgreSQL	70	15.5%	18	245.0
Екатеринбург	1	Go	68	66.7%	13	200.0
Екатеринбург	2	Go + Python	26	25.5%	6	175.0
Екатеринбург	3	Go + Java + Python	10	9.8%	1	150.0

Доли посчитаны от всех вакансий города. В региональной таблице медианы показаны и для малых групп; n зарплат позволяет это увидеть. Полные таблицы с пятью сочетаниями каждого размера — в results/analysis/regions.csv и region_skills.csv.

В Москве 59,6% вакансий варианта и самая высокая медиана — 270 тыс. руб.. В Санкт-Петербурге — 250 тыс. руб., в Екатеринбурге — 195 тыс. руб.. Пара Go + PostgreSQL лидирует в Москве и Санкт-Петербурге: 623 и 164 вакансии, но её доля в Петербурге немного выше — 36,2% против 33,5%.

В Екатеринбурге первая пара — Go + Python: 26 вакансий, 25,5%. Самая частая тройка Go + Java + Python содержит только одну известную зарплату, поэтому её медиану нельзя считать устойчивой. У Москвы и Петербурга лидирует Docker + Go + PostgreSQL; медианы этой тройки — 300 и 245 тыс. руб..

Вывод

MapReduce и DuckDB дали одинаковые частоты для всех трёх размеров стека. DuckDB оказался быстрее, а ранний отбор уменьшил память MapReduce. Самые большие зарплатные различия получились в небольших группах, поэтому их стоит рассматривать вместе с числом наблюдений и регионом.

7. Частоты и SQL расчётов

Ниже три наиболее частых сочетания каждого размера по всему варианту. Частота показывает число вакансий с этим сочетанием; результаты DuckDB полностью совпали.

k	Навыки	Частота
1	Go	2 354
1	PostgreSQL	1 117
1	Docker	716
2	Go + PostgreSQL	1 074
2	Docker + Go	696
2	Go + SQL	545
3	Docker + Go + PostgreSQL	452
3	Go + Kafka + PostgreSQL	302
3	Git + Go + PostgreSQL	298

Расчёт частот в DuckDB

После независимого отбора и нормализации skills содержит уникальные навыки каждой вакансии. Сначала создаю все сочетания, затем считаю частоты:

```
CREATE TEMP TABLE stacks AS
SELECT vacancy_id, [skill] AS stack FROM skills
UNION ALL
SELECT a.vacancy_id, [a.skill, b.skill]
FROM skills a JOIN skills b
  ON a.vacancy_id=b.vacancy_id AND a.skill < b.skill
UNION ALL
SELECT a.vacancy_id, [a.skill, b.skill, c.skill]
FROM skills a JOIN skills b
  ON a.vacancy_id=b.vacancy_id AND a.skill < b.skill
JOIN skills c
  ON a.vacancy_id=c.vacancy_id AND b.skill < c.skill;
SELECT len(stack) AS k, stack, count(*) AS n
FROM stacks GROUP BY stack;
```

Расчёт двух региональных таблиц

```
SELECT area, count(*) AS vacancies, count(salary) AS salary_n,
       median(salary) AS median_salary,
       count(*) FILTER (WHERE len(skills)>0) AS with_skills
FROM selected GROUP BY area
ORDER BY vacancies DESC, area LIMIT 3;

SELECT t.stack, count(*) AS n,
       count(v.salary) AS salary_n, median(v.salary) AS median_salary
FROM stacks t JOIN selected v USING(vacancy_id)
WHERE v.area=? AND len(t.stack)=?
GROUP BY t.stack ORDER BY n DESC, t.stack LIMIT 5;
```

Второй запрос выполняю для каждого из трёх выбранных городов и k=1, 2, 3. В Python добавляю долю: n / общее число вакансий города × 100%. Полный код находится в src/analysis.py, SQL частот — в sql/combinations.sql.